Reviewer Pack — Projective Plane Disjointness Complexity Bounded by Line Cou…

Entry 8b9459572783 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-11 11:36:47 UTC

Projective Plane Disjointness Complexity Bounded by Line Count

Entry ID: `8b9459572783` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-11 11:36:47 UTC

1. Conjecture

Field A (mathematical branch): Finite Geometry (Projective Planes) **Field B** (complexity object): Communication Complexity (Set Disjointness)

Statement:

> For a projective plane of order q, the communication complexity of the set disjointness problem where each player receives a subset of lines and must determine if their sets are disjoint is $\Theta(q^2 \log q)$. This is because the number of lines is $\Theta(q^2)$, and the problem's structure requires checking for intersections, which is inherently tied to the geometry's parameters.

Rationale (proposer's reasoning):

> The projective plane's structure enforces specific intersection properties between lines, which can be leveraged to derive lower bounds on communication complexity. The geometry's parameters directly influence the required information exchange between players.

Taxonomy category: `LIFTING` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `08722c33e21718d4`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM_1	LLM_2
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def is_prime(n):
    if n <= 1:
        return False
    for i in range(2, int(math.sqrt(n)) + 1):
        if n % i == 0:
            return False
    return True

def generate_primes(count):
    primes = []
    num = 2
    while len(primes) < count:
        if is_prime(num):
            primes.append(num)
        num += 1
    return primes

def projective_plane(q):
    points = [i for i in range(q**2 + q + 1)]
    lines = []
    for x in range(q**2 + q + 1):
        line = set()
        for y in range(q**2 + q + 1):
            if (x * y) % (q**2 + q + 1) == 0:
                line.add(x)
                line.add(y)
        lines.append(line)
    return points, lines

def run_trial(seed: int) -> dict:
    random.seed(seed)
    q_values = [2, 3, 4]
    results = []

    for q in q_values:
```

```
        points, lines = projective_plane(q)
        num_lines = len(lines)

        # Simulate communication complexity using a basic protocol
        communication_complexity = 0

        for _ in range(10):  # Sample 10 random subsets of lines for each q
            subset1 = set(random.sample(lines, random.randint(1, num_lines // 2)))
            subset2 = set(random.sample(lines, random.randint(1, num_lines // 2)))

            # Check if the sets are disjoint
            is_disjoint = len(subset1.intersection(subset2)) == 0

            # Simulate communication (each player sends their subset size)
            communication_complexity += 2 * math.ceil(math.log(num_lines, 2))

        results.append({
            "q": q,
            "num_lines": num_lines,
            "communication_complexity": communication_complexity
        })

    mean_communication = sum(result["communication_complexity"] for result in results) / len(resul
    std_communication = math.sqrt(sum((result["communication_complexity"] - mean_communication) **

    conjecture_holds = all(abs(result["communication_complexity"] - (q**2 * math.log(q))) < 10 for
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "Communication Complexity",
        "metric_value": mean_communication,
        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]]
    if not seeds:
        seeds = generate_primes(30)

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_communication = sum(result["metric_value"] for result in results) / len(results)
    std_communication = math.sqrt(sum((result["metric_value"] - mean_communication) ** 2 for resul
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_communication} std={std_communication} support_fracti
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed
```

```
    else:
        print(f"RESULT: INCONCLUSIVE no seeds supported the conjecture")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_08a91518.py", line 96, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_08a91518.py", line 60, in run_trial
    subset1 = set(random.sample(lines, random.randint(1, num_lines // 2)))
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
TypeError: unhashable type: 'set'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed due to unhashable type error, preventing data collection. The conjecture's validity cannot be assessed without successful execution. | next: Fix the TypeError in test code by using frozensets or tuples instead of sets for hashability

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	97470
2	preregistration	ollama_remote	qwen3:8b	0	0	24179
3	novelty	ollama_remote	qwen3:8b	0	0	20661
4	novelty	ollama_remote	qwen3:8b	0	0	15126
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15644
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9995
7	judge	ollama_remote	qwen3:8b	0	0	17490

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 200565 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/8b9459572783.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/8b9459572783.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/8b9459572783.tar.gz` (if generated)